FinMarket-Enterprise

Phase II — Database and Security Integration

Member 1: @errixed — Team Lead, Identity, Authentication, and Infrastructure

Branch: `feature/identity-p2-m1`
Assigned module: `backend/src/modules/identity/`
Additional ownership: Shared infrastructure, Docker, authentication middleware, database configuration, and integration coordination.

---

1. Project Overview

FinMarket-Enterprise is a financial advisory marketing and client acquisition platform designed to help financial advisors manage prospective clients, organize marketing campaigns, deliver authorized marketing communications, automate follow-up activities, and monitor marketing performance.

The application consists of five main modules:

1. Identity and Advisor Management
2. Lead Management (CRM)
3. Marketing Campaigns and Templates
4. Email and Facebook Delivery
5. Automation and Analytics

During Phase I, the backend foundation was established using Node.js and Express.js. Core API routes, initial data models, in-memory repositories, and module-specific business rules were implemented.

Phase II focuses on replacing temporary storage with MongoDB, introducing secure authentication and authorization, enforcing role-based permissions, and establishing persistent relationships between application entities.

The existing backend architecture and API contracts should be preserved wherever practical. Phase I functionality must remain operational after the migration.

1.1 Phase II Objectives

The following requirements must be implemented:

- MongoDB integration using Mongoose.
- Persistent storage through MongoDB-backed repositories.
- Secure user authentication using JSON Web Tokens (JWT).
- Password hashing and credential verification.
- Backend-enforced role-based authorization.
- Secure handling of sensitive information.
- Validated relationships between database collections.
- Dockerized application infrastructure.
- Invitation-based collaboration between Financial Advisors and Marketing Specialists.
- Role selection during user registration.
- Centralized error handling and environment-based configuration.

The backend must contain at least 10 RESTful endpoints, including at least three protected endpoints. The application database must contain at least six collections. The established project design includes eight primary collections, with an additional collection proposed for advisor-specialist assignments.

React frontend integration and deployment remain Phase III responsibilities. Phase II functionality must be testable independently using Postman.

---

2. Team Development and Independence

The existing five-member ownership structure must be maintained.

Member	GitHub Username	Assigned Module
1	@errixed	Identity, Security, Infrastructure

2	@DerenDaniel-Humber	Leads / CRM
3	@KvngWilson	Campaigns / Templates
4	@Bruno-Oliveira2	Delivery
5	@Godwin-pixel	Automation / Analytics

All Phase II branches must be created from the latest `main` branch.

The project infrastructure, including Docker configuration and a working MongoDB environment, must first be prepared by Member 1 and merged into `main`.

After the Dockerized foundation has been merged, each member may clone the repository, create or switch to their respective Phase II branch, and begin implementation.

No member should need to wait for another member's feature implementation to be completed.

To support independent development:

- Database collection names, reference fields, and identifier meanings must be established before module development.
- Authentication and authorization middleware contracts must be documented.
- Modules must use stable shared API contracts and references rather than importing another module's business logic unnecessarily.
- Temporary test fixtures, sample IDs, and mocked authentication contexts may be used during development.
- Each member must independently implement and test the persistence, validation, authorization integration, and business rules belonging to their module.
- Changes to shared infrastructure must be coordinated through Member 1.
- The use of a temporary mock during development must not replace the required real MongoDB and security integration in the completed submission.

The final feature branches must integrate with the shared MongoDB and security infrastructure without requiring major restructuring.

---

3. Shared User Roles and Permissions

The application must support three roles:

3.1 Administrator — `ADMIN`

Administrators are responsible for platform-level access management.

Permissions include:

- Managing user accounts and permissions.
- Managing advisor-specialist relationships when necessary.
- Reviewing or revoking assignments.
- Accessing authorized administrative operations.

Administrator accounts must not be created through ordinary public registration. A controlled administrative provisioning process must be used.

3.2 Financial Advisor — `ADVISOR`

Financial Advisors are the primary owners of prospective client information and marketing campaigns.

Permissions include:

- Managing their own advisor profile.
- Creating and managing their own leads.
- Creating and managing their own campaigns and templates.
- Inviting Marketing Specialists.
- Assigning accepted Marketing Specialists to campaigns.
- Managing their own marketing deliveries and automation activities.
- Viewing analytics associated with their resources.

An advisor must not automatically have permission to access another advisor's private resources.

3.3 Marketing Specialist — `MARKETING_SPECIALIST`

Marketing Specialists perform permitted marketing activities on behalf of Financial Advisors.

Permissions include:

- Maintaining their own user account.
- Receiving, accepting, and declining advisor invitations.
- Viewing campaigns specifically assigned to them.
- Preparing or editing assigned campaigns and templates when authorized.
- Queueing or processing permitted marketing deliveries.
- Viewing authorized campaign performance information.

Marketing Specialists must not automatically access advisor profiles, private lead records, unrelated campaigns, or administrative functions.

A Marketing Specialist must have an accepted advisor relationship and an applicable campaign assignment before accessing that advisor's campaign resources.

The backend must enforce permissions based on both the authenticated role and the relevant resource relationship.

---

4. Database Identifiers and Relationships

The following identifier conventions must be used consistently across all modules.

4.1 Identifier Definitions

Field	Meaning	Referenced Collection
`userId`	Unique identifier of a registered user	`users`
`advisorId`	Unique identifier of an AdvisorProfile, not a User	`advisorProfiles`
`specialistUserId`	User ID belonging to a Marketing Specialist	`users`
`ownerId`	User ID of the owner or creator where specified	`users`
`campaignId`	Unique identifier of a campaign	`campaigns`

templateId	Unique identifier of a marketing template	templates
leadId	Unique identifier of a prospective client	leads
deliveryJobId	Unique identifier of a delivery job	deliveryJobs
automationRuleId	Unique identifier of an automation rule	automationRules
eventId	Unique application-level event identifier used for deduplication	activityEvents

Important distinction:

advisorId always represents AdvisorProfile._id.

It must never be treated as User._id.

For example, an Advisor User can have:

- User._id = A
- AdvisorProfile._id = B
- AdvisorProfile.userId = A

A lead belonging to this advisor must reference advisorId = B, not A.

Similarly, specialistUserId references the specialist's User._id, because Marketing Specialists do not have AdvisorProfile records.

For MongoDB migration, _id should be treated as the persistent Mongoose document identifier. Legacy API identifier formats must be handled consistently so existing references remain functional.

4.2 Required Relationships

Source	Reference Field	Target	Relationship
AdvisorProfile	userId	User	One-to-one
Lead	advisorId	AdvisorProfile	Many-to-one

Campaign	`advisorId`	AdvisorProfile	Many-to-one
Template	`advisorId`	AdvisorProfile	Many-to-one
Campaign	`templateId`	Template	Many-to-one, optional
DeliveryJob	`campaignId`	Campaign	Many-to-one
DeliveryJob	`leadId`	Lead	Many-to-one, optional by channel
AutomationRule	`ownerId`	User	Many-to-one
AutomationRule	`campaignId`	Campaign	Optional reference
ActivityEvent	`ownerId`	User	Many-to-one
AdvisorSpecialistAssignment	`advisorId`	AdvisorProfile	Many-to-one
AdvisorSpecialistAssignment	`specialistUserId`	User	Many-to-one
Campaign	`assignedSpecialistIds`	User	Many-to-many through an array of references

An accepted advisor-specialist relationship authorizes collaboration but does not automatically grant access to every campaign.

Specific campaign access must be determined through campaign assignments.

Mongoose references do not enforce relational integrity automatically. Referenced records, ownership, and permissions must also be validated by backend services.

---

5. Member 1 — Assigned Phase II Responsibilities

5.1 Dockerization and Shared Infrastructure

Owned files: Root Docker configuration, backend Docker configuration, environment templates, and shared database configuration.

The application must be Dockerized before other members begin their Phase II feature implementations.

The Dockerized environment must provide:

1. A Node.js and Express backend container.
2. A MongoDB container.
3. Docker Compose configuration for starting the application.
4. Persistent MongoDB storage using a named Docker volume.
5. Internal container networking.
6. Environment-based configuration.
7. Health checks for backend and database readiness.
8. Reliable startup and shutdown procedures.

Recommended files:

```
None
finmarket-enterprise/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── backend/
    ├── Dockerfile
    ├── .dockerignore
    ├── package.json
    └── src/
```

The backend must connect to MongoDB using its Docker service hostname, not `localhost` inside the container.

Example environment variables:

```
None
PORT=3001
NODE_ENV=development
MONGO_URI=mongodb://mongodb:27017/finmarket_enterprise
JWT_SECRET=replace_with_secure_secret
CORS_ORIGIN=http://localhost:5173
```

The provided values are examples. Real secrets must be configured privately.

The database must remain persistent when the application containers are restarted or recreated without removing the named volume.

The MongoDB port should not be publicly exposed unless required for local development.

The infrastructure must be platform-independent, allowing members to use Docker on Windows, macOS, or Linux.

Completion criteria:

- `docker compose up --build -d` starts the backend and MongoDB.
- The backend establishes a successful MongoDB connection.
- `GET /api/v1/health` responds successfully.
- MongoDB data survives container recreation.
- Environment configuration is documented.
- No real credentials are committed to GitHub.
- The working infrastructure is merged into `main`.

The initial Dockerization must not depend on completion of the other four modules.

5.2 MongoDB Connection and Configuration

Owned files: `backend/src/config/` and shared connection initialization.

A reusable MongoDB connection must be established using Mongoose.

The implementation must:

- Load the MongoDB URI from environment variables.

- Establish the database connection during server initialization.
- Handle connection failures.
- Avoid independently opening database connections in every module.
- Prevent MongoDB credentials from being hardcoded.
- Configure the server to report database connectivity accurately.
- Use a consistent database name across the application.

Each feature module must use Mongoose models and repositories to access the shared database connection.

The existing application architecture must be preserved:

Route → Authentication / Authorization Middleware → Controller → Service → Repository → Mongoose Model → MongoDB

Database queries must not be implemented directly inside route handlers.

5.3 Identity Data Models

Owned directory: `backend/src/modules/identity/`

The existing Identity module must be migrated from in-memory storage to persistent MongoDB storage.

Two primary collections must be implemented:

A. User

Collection: `users`

Required fields should include:

- `_id`
- `name`
- `email`
- `passwordHash`
- `role`
- `isActive`
- `createdAt`
- `updatedAt`

Requirements:

- Email addresses must be normalized.
- Email addresses must be unique.
- Password hashes must never appear in API responses.
- Role values must use the established role constants.
- User activity status must be validated.
- Relevant indexes must be implemented.

B. AdvisorProfile

Collection: `advisorProfiles`

Required fields should include:

- `_id`
- `userId`
- `specialties`
- `consultationTypes`
- `bio`
- `isPublic`
- `createdAt`
- `updatedAt`

Requirements:

- `userId` must reference an existing User with the `ADVISOR` role.
- Each Advisor User must have only one AdvisorProfile.
- Marketing Specialists must not receive AdvisorProfile records.
- Profile fields must retain the validation rules established in Phase I.
- A unique index must prevent multiple profiles for one User.

The migration must preserve the existing registration, login, advisor listing, and profile management APIs wherever practical.

5.4 Registration and Role Selection

The existing registration endpoint must continue to be used:

```
POST /api/v1/auth/register
```

During registration, a user must be able to select one of two permitted roles:

- `ADVISOR`
- `MARKETING_SPECIALIST`

Advisor registration

When `ADVISOR` is selected:

1. Registration data must be validated.
2. The email must be checked for duplicates.
3. The password must be securely hashed.
4. A User record must be created.
5. A related AdvisorProfile must be created.
6. Both records must be persisted successfully.

The creation of the User and AdvisorProfile should be handled atomically where supported, preventing partially completed advisor registrations.

Marketing Specialist registration

When `MARKETING_SPECIALIST` is selected:

1. Registration data must be validated.
2. The password must be securely hashed.
3. A User record must be created.
4. No AdvisorProfile must be created.
5. The specialist must initially have no access to advisor-owned campaigns.

A Marketing Specialist must obtain an accepted advisor relationship and a relevant campaign assignment before managing advisor marketing resources.

The role `ADMIN` must be rejected during ordinary public registration.

5.5 Authentication Using JWT

The existing login endpoint must be updated:

```
POST /api/v1/auth/login
```

Phase I login currently validates credentials without issuing a JWT.

Phase II must implement JWT-based authentication.

The login workflow must:

1. Validate the submitted email and password.
2. Find the user in MongoDB.
3. Verify the stored password hash.
4. Reject invalid credentials and inactive users.
5. Generate a signed JWT for successful authentication.
6. Return the token and permitted public user information.

JWT payloads should include a stable User ID and relevant authorization claims.

A token must have an expiration time.

JWT signing keys must be stored in environment configuration.

Passwords, password hashes, and sensitive credentials must never be included in JWT payloads.

The existing secure `scrypt` implementation may be retained if it is integrated correctly with persistent storage.

A valid token must be provided through the Authorization header:

```
Authorization: Bearer <token>
```

A shared authentication middleware must validate the token and make the authenticated user context available to protected routes.

5.6 Shared Authorization Middleware

Reusable middleware must be implemented for all five modules.

Recommended functionality:

- `authenticate` — Verifies the JWT.
- `authorizeRoles` — Restricts access by user role.
- `requireActiveUser` — Rejects inactive accounts.
- Shared authorization helpers for checking user identity, advisor ownership, accepted assignments, and campaign access.

Middleware names and exports must be documented so the other members can reuse them consistently.

The authenticated request context should expose:

- `userId`
- `role`
- `advisorId`, when the authenticated user is an Advisor
- Other necessary authorization information

The advisor ID must be resolved using AdvisorProfile records.

Important: Client-submitted identifiers must not be trusted as proof of identity.

For example, an advisor must not be able to update another advisor's campaign by simply changing `advisorId` in the request body.

Resource ownership and role permissions must be verified on the backend.

An administrator must have only explicitly granted administrative permissions. The `ADMIN` role must not automatically bypass every resource ownership rule without an intentional policy.

5.7 Invitation-Based Advisor and Specialist Relationships

An invitation-based collaboration system must be implemented.

The system must allow an Advisor to invite an existing Marketing Specialist, who may accept or decline the invitation.

AdvisorSpecialistAssignment Model

A new collection is recommended:

`advisorSpecialistAssignments`

Suggested fields:

- `_id`
- `advisorId`
- `specialistUserId`
- `status`

- `createdAt`
- `updatedAt`
- `acceptedAt`
- `revokedAt`

Allowed statuses:

- `PENDING`
- `ACCEPTED`
- `DECLINED`
- `REVOKED`

The following rules must be enforced:

1. Only authenticated Advisors may create invitations for their own AdvisorProfile.
2. The invited user must exist and have the `MARKETING_SPECIALIST` role.
3. A specialist cannot invite themselves on behalf of an advisor.
4. Only the invited specialist may accept or decline the invitation.
5. Duplicate active invitations or assignments between the same advisor and specialist must be prevented.
6. An invitation must be accepted before collaboration permissions become available.
7. Advisors must be able to revoke their own specialist relationships.
8. Revoked relationships must immediately prevent further access to the affected advisor's resources.
9. Administrative overrides, if implemented, must require the appropriate administrative permission.

An Advisor may collaborate with multiple specialists.

A Marketing Specialist may collaborate with multiple Advisors.

The relationships must remain explicitly recorded and independently revocable.

Suggested API endpoints

Method	Endpoint	Purpose
POST	`/api/v1/advisor-invitations`	Create invitation
GET	`/api/v1/advisor-invitations`	List relevant invitations

POST	`/api/v1/advisor-invitations/:id/accept`	Accept invitation
POST	`/api/v1/advisor-invitations/:id/decline`	Decline invitation
GET	`/api/v1/advisor-assignments`	List accepted relationships
DELETE	`/api/v1/advisor-assignments/:id`	Revoke relationship

All endpoints must be protected.

The backend must derive the caller's identity from the authenticated context.

Only invitations and assignments associated with the authenticated user may be returned, except when explicitly permitted for administrators.

The creation and management of campaign-specific specialist assignments belong to Member 3.

Member 1 must provide the shared relationship lookup and authorization contracts needed by Member 3, without implementing the Campaigns module.

5.8 Secure Data Handling

The Identity module and shared backend security components must implement:

- Request input validation.
- Secure password hashing.
- Duplicate-email prevention.
- Password hash exclusion from API responses.
- JWT expiration and signature verification.
- Inactive-account restrictions.
- Role-based permission checks.
- Resource ownership validation.
- Secure environment variable handling.
- Centralized error handling.
- Consistent HTTP response statuses.
- Protection against unauthorized modification of roles and account information.

Account registration must not allow clients to submit privileged fields such as `passwordHash`, `isActive`, or unauthorized administrative permissions.

Public advisor directory endpoints should return only information intended for public visibility.

MongoDB or internal server errors must not expose credentials, connection strings, or sensitive implementation details in API responses.

5.9 API Compatibility

The following Phase I endpoints must remain functional:

Method	Endpoint
POST	`/api/v1/auth/register`
POST	`/api/v1/auth/login`
GET	`/api/v1/advisors`
GET	`/api/v1/advisors/:id`
PATCH	`/api/v1/advisors/:id`

Authentication responses must be updated to include JWT-based authentication.

Profile modification must require authentication and authorization.

Public advisor listing and public profile retrieval may remain accessible without authentication, subject to profile visibility settings.

Existing endpoint response structures should be preserved where possible. Any necessary changes must be documented for the other team members.

---

6. Shared Security and Integration Contracts

To ensure all members can work independently, the following contracts must be established and documented.

Authentication contract

All protected modules must use the same JWT authentication middleware and authenticated user context.

Authorization contract

All modules must use the same role constants and interpret `advisorId` consistently.

The shared authorization layer must provide a way to determine:

- Whether a user is authenticated.
- Whether a user has an allowed role.
- Whether an advisor owns a resource.
- Whether an advisor-specialist relationship is accepted.
- Whether the specialist has access to the relevant advisor.
- Whether a campaign-specific assignment is required.

Campaign-specific assignment enforcement remains the responsibility of Member 3.

Database contract

Every module must use the same Mongoose connection and persistent identifier conventions.

References between collections must be validated, even when related records belong to another member's module.

Error handling contract

A consistent error response format must be established.

Recommended structure:

```
None
{
  "error": {
    "code": "FORBIDDEN",
    "message": "You do not have permission to perform this
action.",
    "details": []
  }
}
```

Recommended HTTP statuses:

- `400` — Invalid request.
- `401` — Authentication required or invalid credentials.
- `403` — Authenticated but not authorized.
- `404` — Resource not found.
- `409` — Duplicate record or conflicting operation.
- `500` — Internal server error.
- `503` — Required service temporarily unavailable.

Shared middleware and interface documentation must be made available to the other members after the infrastructure foundation is merged.

---

7. Testing Requirements

The Identity module and shared security infrastructure must be tested using Postman and appropriate automated tests.

At minimum, the following scenarios must be verified:

Database and Docker

- Backend connects to MongoDB.
- Backend starts correctly through Docker Compose.
- MongoDB data persists after container recreation.
- Missing or invalid database configuration is handled.

- Database connection failures do not produce misleading healthy responses.

Registration

- Advisor registration succeeds.
- AdvisorProfile is created with the correct `userId`.
- Marketing Specialist registration succeeds.
- Marketing Specialist registration does not create an AdvisorProfile.
- Duplicate email registration is rejected.
- Invalid registration information is rejected.
- Public `ADMIN` registration is rejected.
- User records remain available after application restart.

Authentication

- Correct credentials return a valid JWT.
- Incorrect credentials are rejected.
- Invalid JWT is rejected.
- Expired JWT is rejected.
- Missing JWT is rejected on protected routes.
- Inactive users cannot access protected resources.
- Password hashes are never returned.

Authorization

- Advisors can modify their own profiles.
- Advisors cannot modify other advisors' profiles.
- Specialists cannot modify advisor profiles.
- Specialists cannot access unrelated advisor resources.
- Privileged operations reject unauthorized roles.

Invitations

- Advisor can invite a registered specialist.
- Specialist can accept an invitation addressed to them.
- Specialist can decline an invitation.
- Unrelated users cannot accept invitations.
- Duplicate active invitations are rejected.
- Accepted assignments are persisted.
- Advisor can revoke an assignment.
- Revoked assignments no longer authorize access.

All tests must use realistic sample records and valid database references.

---

8. Documentation and GitHub Requirements

The following documentation must be maintained:

- Docker setup instructions.
- MongoDB environment configuration.
- Application startup and shutdown commands.
- Database collection definitions.
- Identifier and relationship conventions.
- Registration and login behavior.
- JWT authentication usage.
- Role and permission definitions.
- Invitation and assignment workflows.
- Protected API examples.
- Postman test instructions.
- Known integration assumptions.

The `.env.example` file must contain placeholder values only.

Actual credentials and environment secrets must not be committed.

Development must occur in:

`feature/identity-p2-m1`

Code changes must be committed with meaningful messages and submitted through a pull request targeting `main`.

The individual contribution must remain identifiable through GitHub history.

Shared infrastructure changes should be merged early, while Identity feature development can continue independently on the assigned branch.

---

9. Completion Criteria

Member 1's Phase II responsibilities will be considered complete when:

- The backend and MongoDB are Dockerized and start successfully.
- MongoDB persistence works across application restarts.
- Shared MongoDB configuration is available to all modules.
- User and AdvisorProfile models use persistent MongoDB storage.
- Registration supports Advisor and Marketing Specialist role selection.
- Public Administrator registration is blocked.
- Advisor registration creates a related AdvisorProfile.
- Marketing Specialist registration creates a User without AdvisorProfile.
- Passwords are securely hashed and verified.
- Login returns valid signed JWTs.
- Shared authentication and role-based authorization middleware is implemented.
- Advisor ownership checks use AdvisorProfile IDs correctly.
- Advisor-specialist invitations can be created, accepted, declined, and revoked.
- Assignment records persist in MongoDB.
- Unauthorized and unassigned users are denied protected access.
- Existing Identity endpoints remain functional where applicable.
- Shared contracts and error handling are documented.
- Postman and automated tests cover successful and unsuccessful operations.
- Docker and infrastructure setup instructions are provided.
- Changes are committed and submitted through GitHub.

Expected outcome: A Dockerized and secure backend foundation will be available, with persistent Identity records, JWT-based authentication, backend-enforced role permissions, and invitation-based Advisor–Marketing Specialist relationships. Other modules will be able to integrate through the documented shared contracts without changing their independently owned business logic.